

GOLDEN PATH STACK BLUEPRINT

Standardized tech decisions for every new project

58 Decisions | 93% Coverage | 12 Categories

Generated: 2026-04-03

Executive Summary

This document captures every technology decision made for the golden path stack. It is the single source of truth for starting new projects. Instead of re-researching tools and re-making decisions, clone the template repo and start building.

Core Platform Pillars

- **Runtime & Language:** Bun + TypeScript everywhere. One tool for package management, bundling, testing, and execution. Acquired by Anthropic, powers Claude Code.
- **Cloud & Compute:** GCP with Cloud Run (per-request billing, scale to zero). Pulumi for all infrastructure as TypeScript code. Pulumi ESC for secrets.
- **Backend Platform:** Supabase for database (Postgres), auth, file storage, vector store (pgvector), realtime, queues, cron, and edge functions.
- **API Layer:** tRPC for end-to-end type-safe internal APIs. Zod for all schema validation. No OpenAPI/Swagger needed for internal contracts.
- **Frontend:** Next.js (React) for web, React Native + Expo for mobile, Electron for desktop. Same React/TS stack across all platforms. Tailwind CSS + Shadcn/ui for components. Zustand + TanStack Query for state.
- **Observability:** PostHog for all 12 products: product analytics, web analytics, session replay, feature flags, experiments, error tracking, surveys, data warehouse, CDP, LLM analytics, revenue analytics, and workflows.
- **Payments:** Stripe (processor) + RevenueCat (subscription management) for unified cross-platform billing across iOS, Android, and web.
- **AI Stack:** OpenRouter as LLM gateway (300+ models, no vendor lock-in). Vercel AI SDK + @openrouter/ai-sdk-provider for frontend streaming UI. Claude Code SDK / OpenCode for agent runtimes. Gemini embeddings stored in Supabase pgvector. PostHog LLM Analytics for observability.
- **Security & Edge:** Cloudflare free tier for CDN, SSL, DDoS protection, WAF. GoDaddy for domains, nameservers pointed to Cloudflare.
- **Dev Tooling:** Bun workspaces for monorepo. GitHub Actions for CI/CD. Biome for linting/formatting. Bun test + Playwright for testing. Changesets for versioning.

1. Infrastructure & Cloud

Category	Choice	Status	Notes
Cloud Provider	GCP	✓ Decided	Cloud Run for APIs, Compute Engine for persistent
IaC	Pulumi (TypeScript)	✓ Decided	All infra as TS code. State in Pulumi Cloud.
Containers	Docker	✓ Decided	Dockerfile per service (Bun base), compose for local
Container Registry	GCP Artifact Registry	✓ Decided	Private images, same GCP project
Managed Compute	Cloud Run (per-request)	✓ Decided	Scale to zero. Free tier friendly.
DNS / Domains	GoDaddy + Cloudflare	✓ Decided	Domains on GoDaddy, NS to Cloudflare
CDN / Security	Cloudflare (free tier)	✓ Decided	Global cache, SSL, DDoS, WAF
Secrets	Pulumi ESC	✓ Decided	Replaces GCP Secret Manager (\$80/mo savings)

Rationale

Cloud Run per-request billing: Falls under free tier for low-traffic services. You pay nothing when idle. Bun's ~50ms cold starts make this especially cost-effective.

Cloudflare: A free security + performance layer between the internet and your GCP backend. Caches static assets on 300+ global edge servers, provides free SSL certificates, blocks DDoS attacks, and offers a WAF. Setup is just pointing GoDaddy nameservers to Cloudflare.

Pulumi ESC: GCP Secret Manager was costing ~\$80/month just for stored secrets with disorganized config. Pulumi ESC centralizes environment and secret configurations per Pulumi stack, integrates natively with Pulumi IaC, and eliminates the separate billing.

2. Backend & APIs

Category	Choice	Status	Notes
Language	TypeScript (everywhere)	✓ Decided	Frontend, backend, IaC, scripts
Runtime + Pkg Mgr	Bun	✓ Decided	All-in-one: runtime, pkg mgr, bundler, test runner
API Framework	tRPC	✓ Decided	E2E type safety for internal APIs
DB Client	Supabase JS Client	✓ Decided	@supabase/supabase-js for everything
Validation	Zod	✓ Decided	Schema validation everywhere, pairs with tRPC
Jobs & Queues	Supabase Queues + Cron	✓ Decided	Built into Supabase platform

Why Bun Over Node.js

- 2.4x faster HTTP throughput (benchmarked with Hono)
- 30-40% less memory usage at idle and under load
- Native TypeScript execution — no ts-node, no Babel, no build step
- Built-in package manager (6-9x faster than npm), bundler, and test runner
- 98% Node.js API compatibility in 2026 — most npm packages work unchanged
- Acquired by Anthropic in December 2025, powers Claude Code. Long-term stability guaranteed.
- Official Cloud Run deployment support with Bun Docker base images
- Replaces 3 separate tools: Node.js runtime + pnpm package manager + Vitest test runner

Why tRPC Over REST/OpenAPI

All API contracts are internal (frontend calling own backend). tRPC eliminates the need for OpenAPI schemas, Swagger docs, and code generation. You define a function on the backend, call it on the frontend, and TypeScript validates the entire contract at compile time. For any external APIs exposed to customers later, a Hono REST layer can be added alongside tRPC.

Caching Decision: Skip For Now

A caching layer (like Redis) sits between your app and database to serve frequently-accessed data faster. At current scale this adds complexity without meaningful benefit. Supabase handles connection pooling internally. When scale demands it, add Upstash Redis (serverless Redis, pay-per-request).

3. Data & Storage (Supabase Platform)

Category	Choice	Status	Notes
Database	Supabase Postgres	✓ Decided	Managed Postgres, RLS, migrations via CLI
Authentication	Supabase Auth	✓ Decided	Email/pass, OAuth, magic links, JWTs, RLS
File / Media Storage	Supabase Storage	✓ Decided	Images, videos, files. S3-compatible. RLS.
Vector Store	Supabase Vector (pgvector)	✓ Decided	Embeddings via Gemini, stored in pgvector
Realtime	Supabase Realtime	✓ Decided	WebSocket subscriptions, presence, broadcast
Edge Functions	Supabase Edge Functions	✓ Decided	Deno serverless for Supabase logic
Backup / Bulk	GCS (Cloud Storage)	✓ Decided	Backups, exports, large assets

Why Supabase for Everything

Supabase consolidates 7+ separate tool decisions into one platform: database, auth, file storage, vector store, realtime, queues/cron, and edge functions. This means one SDK (@supabase/supabase-js), one dashboard, one billing relationship, and one set of docs. The underlying database is standard Postgres, so there's no vendor lock-in on the data layer. Row-Level Security (RLS) policies provide per-user and per-tenant data isolation without application-level authorization code.

4. Frontend & UI

Category	Choice	Status	Notes
Web Framework	Next.js (React)	✓ Decided	App Router, RSC. Cloud Run or Vercel.
Mobile Framework	React Native + Expo	✓ Decided	Shared React model. Expo for builds/OTA.
Desktop Framework	Electron	✓ Decided	Native desktop apps. Same React/TS stack. AI SDK works here too.
Styling / Components	Tailwind CSS + Shadcn/ui	✓ Decided	Industry standard. Copy-paste components.
State Management	Zustand + TanStack Query	✓ Decided	Client state + server/async state

State Management Strategy

Zustand: Lightweight client-side state (UI state, local preferences, modal state). Simple API, no boilerplate, works with React DevTools.

TanStack Query: Server state management (API data, caching, background refetching, optimistic updates). Eliminates manual loading/error states and keeps server data fresh automatically. Pairs naturally with tRPC.

5. CI/CD & Dev Tooling

Category	Choice	Status	Notes
Repo Strategy	Monorepo (Bun workspaces)	✓ Decided	Shared packages across apps
CI/CD	GitHub Actions	✓ Decided	Standard, huge marketplace, generous free tier
Linting / Formatting	Biome	✓ Decided	Replaces ESLint + Prettier. ~100x faster.
Testing	Bun test + Playwright	✓ Decided	Built-in unit runner + E2E
Environments	dev / prod	✓ Decided	Pulumi stacks per env + Pulumi ESC
Versioning	Changesets	✓ Decided	Auto CHANGELOG.md from PRs

Monorepo Structure

Every new project is a self-contained Bun workspace monorepo:

```
my-project/
├── package.json           ← workspace root
├── CLAUDE.md              ← global AI agent context
├── apps/
│   ├── web/              ← Next.js app
│   ├── mobile/           ← React Native + Expo
│   ├── desktop/          ← Electron app
│   └── api/               ← tRPC API server
├── packages/
│   ├── ui/               ← shared Shadcn components
│   ├── db/               ← Supabase client + types
│   └── config/            ← shared tsconfig, biome config
└── infra/                ← Pulumi stacks
```

Shared Packages Across Repos

For code that is universal across ALL projects (not just apps within one monorepo), create a separate shared packages repo and publish to GitHub Packages (free for private packages):

```
@yourorg/analytics    ← PostHog wrapper
@yourorg/supabase     ← configured Supabase client
@yourorg/config        ← shared tsconfig, biome, tailwind
@yourorg/types         ← shared type definitions
```

Any project repo can then install these as normal dependencies:

```
bun add @yourorg/analytics @yourorg/supabase
```

Update the shared package once, publish a new version, and all consuming repos pick up the change on their next install. This is the recommended approach for things like an analytics client wrapper that should be identical across 20 different projects.

6. PostHog (Product OS) — All 12 Products

PostHog replaces an entire category of SaaS tools. Every product below is used, with generous free tiers that reset monthly:

Category	Choice	Status	Notes
Product Analytics	PostHog	✓ Decided	Events, funnels, retention, cohorts, HogQL. 1M events/mo free.
Web Analytics	PostHog	✓ Decided	GA replacement. Pageviews, sources, web vitals.
Session Replay	PostHog	✓ Decided	Watch real sessions. 5K recordings/mo free.
Feature Flags	PostHog	✓ Decided	Roll out to segments. 1M requests/mo free.
A/B Experiments	PostHog	✓ Decided	Statistical significance testing. No-code.
Error Tracking	PostHog	✓ Decided	Exceptions, alerts. 100K/mo free.
Surveys	PostHog	✓ Decided	In-app NPS, feedback. 1500/mo free.
Data Warehouse	PostHog	✓ Decided	HogQL (SQL), 7yr retention, external sources.
CDP	PostHog	✓ Decided	60+ sources. Transforms, GeoIP, PII scrub.
LLM Analytics	PostHog	✓ Decided	Traces, latency, token cost. Own free tier.
Revenue Analytics	PostHog	✓ Decided	Transaction values, LTV, sub changes.
Workflows	PostHog	✓ Decided	Automate actions on user behavior.

The CDP (Customer Data Platform) connects PostHog to external data sources like Stripe, HubSpot, Salesforce, and 60+ others. This means PostHog becomes the single warehouse for product data AND business data, queryable via SQL (HogQL). LLM Analytics is particularly relevant for AI-powered products — it tracks token usage, model performance, latency, and cost per conversation.

7. Payments & Billing

Category	Choice	Status	Notes
Payment Processor	Stripe	✓ Decided	Card processing. Used by RevenueCat under the hood.
Subscription Mgmt	RevenueCat	✓ Decided	iOS + Android + Web. Entitlement sync.
Mobile IAP	RevenueCat	✓ Decided	StoreKit + Google Play. Paywall A/B testing.
Web Billing	RevenueCat Web Billing	✓ Decided	RC's billing engine. No-code checkout, web SDK.

How Stripe + RevenueCat Work Together

Stripe handles the actual payment processing (card charges, refunds, etc). RevenueCat sits on top as the subscription lifecycle manager. It handles entitlements (which features a user has access to), cross-platform syncing (a user who pays on iOS can access premium features on web), paywall A/B testing, cohort analytics, and the billing UI.

RevenueCat Web Billing is their own billing engine for web purchases. It uses Stripe as the payment processor underneath but manages the subscription logic itself. This means you use RevenueCat as the single abstraction layer across all platforms instead of managing Stripe Billing separately for web and RevenueCat separately for mobile.

8. Communications

Category	Choice	Status	Notes
Transactional Email	Resend + React Email	✓ Decided	Modern API + React component templates
Notifications	PostHog Workflows	✓ Decided	Behavioral triggers. Add Novu later if needed.

Resend is a modern email API with excellent developer experience. React Email lets you build email templates using React components and Tailwind CSS — the same tools you use for your web frontend. This means no context-switching to learn MJML or HTML email quirks.

For notifications triggered by user behavior (user hasn't logged in for 7 days, user completed onboarding, etc), PostHog Workflows handles this natively since PostHog already has all the behavioral data. For multi-channel notifications (email + push + in-app + SMS), add Novu when that complexity is needed.

9. AI & LLM Stack

Category	Choice	Status	Notes
LLM Gateway	OpenRouter	✓ Decided	Single API for 300+ models. No vendor lock-in.
AI Frontend SDK	Vercel AI SDK + @openrouter/ai-sdk-provider	✓ Decided	useChat, streamText, tool-use UI via OpenRouter
Agent Framework	Claude Code SDK / OpenCode	✓ Decided	Agent provisioning and execution runtime
Embeddings Model	Gemini Embedding	✓ Decided	Google text-embedding-004. Stored in pgvector.
Vector Store	Supabase Vector (pgvector)	✓ Decided	Same Postgres instance. No separate DB.
LLM Observability	PostHog LLM Analytics	✓ Decided	Traces, latency, cost, quality tracking
AI Coding Agent	Claude Code	✓ Decided	Primary dev tool. Runs on Bun.

Four-Layer AI Architecture

The AI stack has four distinct layers with no overlap:

Layer 1 — LLM Gateway (backend): OpenRouter provides a single API that routes to Claude, GPT-4, Gemini, Llama, Mistral, and 300+ other models. You get model flexibility, automatic fallback routing if a provider is down, and no vendor lock-in. One API key, one SDK, any model.

Layer 2 — Frontend Streaming (UI): Vercel AI SDK provides React hooks (useChat, useCompletion) for streaming LLM responses, rendering tool calls, displaying structured objects, and managing conversation state. The @openrouter/ai-sdk-provider package plugs OpenRouter directly into the AI SDK, so all streaming flows through your OpenRouter gateway. Works in Next.js, React Native, and Electron. This handles all the painful frontend plumbing for AI-powered UIs: token streaming, tool-use rendering, multi-step agent displays, etc.

Layer 3 — Agent Runtime (backend): Claude Code SDK and/or OpenCode for provisioning and executing actual AI agents in your products. These are the runtimes that do the agentic work — tool calling, multi-step reasoning, code execution. This is distinct from the LLM gateway (which just routes API calls) and the frontend SDK (which just renders the output).

Layer 4 — Observability: PostHog LLM Analytics tracks every LLM interaction: traces, generations, latency, token cost, and conversation quality. This feeds into the same PostHog dashboard as all your other product analytics, so you can correlate AI usage with user behavior and business outcomes.

10. Documentation & DX

Category	Choice	Status	Notes
In-Repo Docs	Markdown in repo	✓ Decided	README, CONTRIBUTING, docs/ folder
API Docs (internal)	tRPC Panel	✓ Decided	Auto-generated from tRPC router
Product Docs	TBD	? Open	Fumadocs or Mintlify when needed
Changelog	Changesets	✓ Decided	Auto CHANGELOG.md from PRs

At current scale, documentation lives in the repo as markdown files. This is version controlled, searchable in GitHub, and requires zero additional tooling. tRPC Panel auto-generates an interactive API explorer from your tRPC router definition — no Swagger, no code generation, no maintenance.

Customer-facing product documentation is not needed until there are customers using your product. When that time comes, Fumadocs (open source, Next.js native) or Mintlify (hosted, polished) are the recommended options.

11. Legal & Compliance

Category	Choice	Status	Notes
Privacy Policy / ToS	TBD	? Open	Termly or lawyer when needed
Cookie Consent	TBD	? Open	PostHog respects DNT. Banner may be needed.
GDPR / Data Residency	TBD	? Open	Supabase EU + PostHog EU available

These decisions depend on what markets you serve and when you have paying customers. Both Supabase and PostHog offer EU-hosted options for GDPR compliance. Termly can auto-generate compliant privacy policies and cookie consent banners. These should be addressed before launching to customers but do not block technical development.

12. CLAUDE.md Hierarchy for AI Coding Agents

Claude Code reads CLAUDE.md files hierarchically based on the working directory. This means every level of the monorepo can have its own agent context that stacks on top of the global rules.

How It Works

When Claude Code is launched from a directory, it walks up the directory tree and loads every CLAUDE.md it finds. If you run “cd apps/web && claude”, it loads:

my-project/CLAUDE.md	← global rules (ancestor)
my-project/apps/CLAUDE.md	← app conventions (ancestor)
my-project/apps/web/CLAUDE.md	← Next.js specifics (cwd)

When launched from the repo root, Claude Code loads the root CLAUDE.md and discovers sub-directory context as it navigates into files during a task.

What Goes in Each Level

Root CLAUDE.md: Golden path enforcement. Stack decisions (Bun, Supabase, tRPC, Zod, Biome), global conventions (TypeScript strict mode, always validate with Zod, import shared packages from @yourorg/*), error handling patterns, naming conventions, banned libraries (no axios, no lodash, no moment).

packages/ CLAUDE.md: These are published packages — don’t break the public API. Export patterns, testing requirements, semver rules.

apps/web/ CLAUDE.md: Next.js App Router, use server components by default, never use pages directory, styling is Tailwind + Shadcn only, state management is Zustand + TanStack Query.

apps/api/ CLAUDE.md: Every tRPC procedure gets a Zod input schema. All mutations are protected procedures. Database access only through Supabase JS client.

apps/mobile/ CLAUDE.md: React Native + Expo. Use Expo modules, not bare RN APIs. Test on iOS and Android. No web-only APIs.

apps/desktop/ CLAUDE.md: Electron with React renderer. Share components from packages/ui. Use IPC for main/renderer process communication. Vercel AI SDK works here for streaming AI UIs.

infra/ CLAUDE.md: Pulumi TypeScript only. Target GCP. Cloud Run for APIs. Use Pulumi ESC for secrets. Never hardcode project IDs or credentials.

The template repo ships with all these CLAUDE.md files pre-written. Every new project starts with the AI coding agent pre-configured at every level from day one.

Next Steps: Implementation Plan

Phase 1: Shared Packages Repo

- Create a @yourorg/packages repo with Bun workspaces
- Build the PostHog analytics wrapper (@yourorg/analytics)
- Build the configured Supabase client wrapper (@yourorg/supabase)
- Build shared config package (tsconfig, biome, tailwind presets)
- Publish to GitHub Packages (private npm registry, free)

Phase 2: Template Repo

- Scaffold the monorepo structure (apps/web, apps/mobile, apps/desktop, apps/api, packages/ui, infra/)
- Pre-wire all configs: biome.json, tsconfig, tailwind, Docker, GitHub Actions
- Set up tRPC router with Zod validation in apps/api
- Set up Next.js with Tailwind + Shadcn + Zustand + TanStack Query in apps/web
- Set up Supabase project init (migrations, RLS policies, storage buckets)
- Set up Pulumi stacks for dev/prod with Cloud Run + Artifact Registry
- Pre-configure PostHog SDK initialization
- Pre-configure RevenueCat SDK initialization
- Pre-configure Vercel AI SDK + OpenRouter provider
- Pre-configure Resend + React Email templates
- Write all CLAUDE.md files at every directory level
- Mark repo as GitHub template

Phase 3: First Project

- Create new repo from template
- Install shared packages from @yourorg/*
- Customize for project-specific needs
- Deploy via Pulumi
- Start building features immediately